A Learned Fixed-Size State for Long-Horizon Computer-Use Agents

Pengfei Ma

1 Problem and Gap

Long-horizon computer-use agents (CUAs) must preserve task constraints across many steps while revising their understanding when the interface changes. OSWorld 2.0 identifies long-horizon state drift, stale grounding, and stale clicks as recurring failure modes [1]. Qwen-CUA reports 86.2 on OSWorld-Verified and, separately, 18.5 binary / 48.4 partial completion on OSWorld 2.0 [2]. These come from different benchmarks and metrics and are not a direct comparison, but they indicate the difficulty of long-horizon computer use. Recent surveys likewise place long-horizon problems among the field's central bottlenecks [3]. Existing systems manage history through mechanisms such as a bounded screenshot window, structured textual memory, or deterministic folding of older observations [4, 2]. These are useful engineering choices. They motivate a narrower question: under a fixed state budget, can an agent learn which task constraints to retain and which new observations should revise its working state? This question is consistent with calls for history simplification to become a learnable part of computer-use agents [5].

2 Proposed Idea

I propose adding a learned fixed-size task state to an existing CUA. The agent's current vision encoder would embed each screenshot as e_t; a recurrent state $s_t \in \mathbb{R}^d$ would supplement, not replace, the current screenshot and short visual history. The state is intended to summarize persistent task constraints, completed subgoals, and unresolved items. A small adapter would expose it to the controller as compact memory tokens.

Following the neural memory ODE formulation [6], the state evolves under the current observation:

$$\dot{s}(\tau) = -s(\tau) + f\big(s(\tau) + g(e_t)\big), \tag{1}$$

where g projects the screenshot embedding into state space and f is a learned nonlinear write map. The decay term provides a path for state change, while the observation-conditioned write map lets new evidence interact with the existing state.

Let $F(s; e) = -s + f\big(s + g(e)\big)$. The formulation's existing discrete implementations advance the state with a first-order Euler step [7]; I would instead use a Heun-style predictor-corrector:

$$\begin{aligned} \tilde{s}_{t+1} &= s_t + \Delta t_t \odot F(s_t; e_t), \\ s_{t+1} &= s_t + \frac{\Delta t_t}{2} \odot \left[F(s_t; e_t) + F(\tilde{s}_{t+1}; e_t)\right], \end{aligned} \tag{2}$$

with a per-dimension step $\Delta t_t = \sigma(a_\theta([e_t; s_t])) \in (0, 1)^d$ that conditions on both the observation and the current state, so the update rate can reflect how the two relate rather than the observation alone. For each dimension i, the coefficient on the linear decay path is

$$1 - \Delta t_{t,i} + \frac{\Delta t_{t,i}^2}{2} \in \left(\frac{1}{2}, 1\right).$$

For the isolated linear decay component, this is Heun's stability factor and lies in a conservative stable range. This does not guarantee stability of the full nonlinear dynamics, which also depends on f; nor does the coefficient alone guarantee a particular forgetting rate, because the write terms also depend on the previous state. The bounded step is instead a design choice for controlling the update rate.

The corrector evaluates the vector field at both the current and predicted states, so the update can respond to how that field varies across one step. My working hypothesis is that this variation may help the state

respond when new visual evidence conflicts with its current contents. The predictor and corrector use the same screenshot embedding, so the projection $g(e_t)$ can be shared rather than recomputed. My first-author work, GhostODE, developed a shared-projection Heun update for compact vision networks [8]; applying that efficiency idea to temporal task-state maintenance is the proposed transfer, not a result already validated for CUAs. Selective state-space models such as Mamba also have input-dependent state updates [9], so the intended distinction is the second-order update and shared observation projection, not learnable time scales alone. The state also cannot recover system information that is never visible in the GUI, since the interface is only an incomplete projection of the system state [10].

References

[1] M. Yuan, Z. Zhou, X. Xiong, et al. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint* arXiv:2606.29537, 2026.

[2] D. Lu, S. Bai, T. Bai, et al. Qwen-CUA: Native computer use for (almost) everything. *arXiv preprint* arXiv:2608.02352, 2026.

[3] K. Marino, M. F. Ishmam, and A. Marasović. Computer use survey — a visual survey of computer use agents. In *ICLR Blogposts*, 2026. iclr-blogposts.github.io

[4] X. Wang, B. Wang, D. Lu, et al. OpenCUA: Open foundations for computer-use agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025. doi:10.52202/085713-4669

[5] P. J. Sager, B. Meyer, P. Yan, et al. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *Journal of Artificial Intelligence Research*, 85:34, 2026. doi:10.1613/jair.1.19490

[6] Z. Yi. nmODE: Neural memory ordinary differential equation. *Artificial Intelligence Review*, 56(12):14403–14438, 2023. doi:10.1007/s10462-023-10496-2

[7] L. Yu, J. Wu, B. Gou, X. Min, L. Zhang, Z. Yi, and T. He. MobileODE: An extra lightweight network. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, pp. 120931–120956, 2025. proceedings.neurips.cc | doi:10.52202/085713-4034

[8] P. Ma, L. Yu, L. Zhang, Z. Yi, and T. He. GhostODE: A lightweight network optimized by ordinary differential equation. *Neurocomputing*, 2026. doi:10.1016/j.neucom.2026.134898

[9] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint* arXiv:2312.00752, 2023.

[10] Y. Wu and Y. Yang. A platform-aware survey of execution, verification, and risk of graphical user interface agents. *ENGINEERING Information Technology & Electronic Engineering*, 27(7):1–26, 2026. jzus.zju.edu.cn | IEEE Xplore